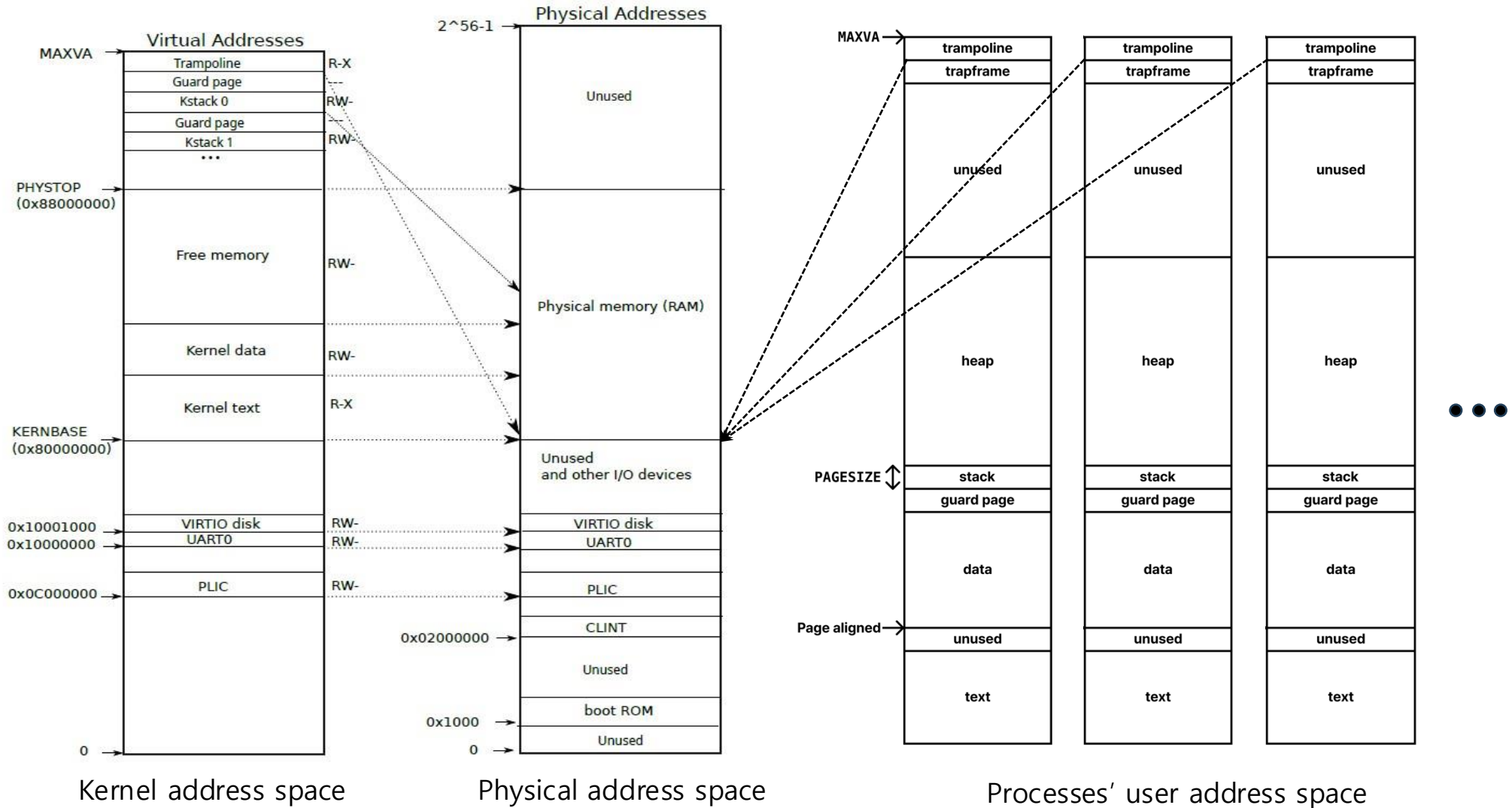

Virtual Memory (2)

More detail about user virtual memory(uvm)

Recap: Previous Lab Session

- Overview
 - Virtual memory
 - Paging
- Page table
 - User page table creation
 - Kernel page table creation and initialization
 - Page table switching

Recap: Previous Lab Session



Contents

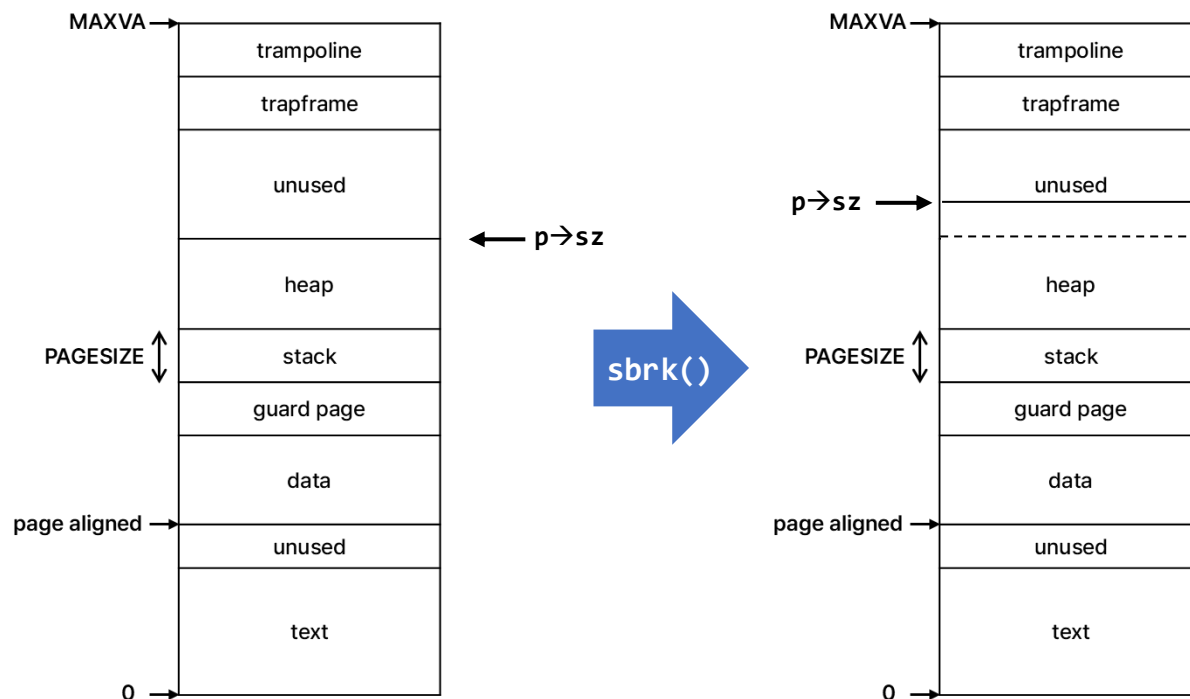
- User-Level Memory Allocation APIs
- sbrk System Call
- Core VM Helper Functions User-Level Memory Allocation
 - uvmalloc
 - uvmdealloc
 - uvmunmap
 - uvmcopy
 - ...

User-Level Memory Allocation

- Classic user memory allocation APIs
 - From "The C Programming Language", Kernighan and Ritchie
 - Divide heap memory space into small sized (16 Byte) unit
 - Manage available memory units using circular single linked list
- **malloc(nbytes)**: Allocates memory using first-fit search
- **free(ptr)**: Returns memory to free list with automatic coalescing
- **morecore(units)**: Expands heap using **sbrk()** when needed (min 4096 units = 64KB)

sbrk

- sbrk system call is used to dynamically adjust heap memory of the process.
- Provides lazy allocation which only updates the metadata pointer and defers physical allocation.



```
$ vim kernel/sysproc.c

39  uint64
40  sys_sbrk(void)
41  {
42      uint64 addr;
43      int t;
44      int n;
45
46      argint(0, &n);
47      argint(1, &t);
48      addr = myproc()->sz;
49
50      if(t == SBRK_EAGER || n < 0) {
51          if(growproc(n) < 0) {
52              return -1;
53          }
54      } else {
55          ...
62          myproc()->sz += n;
63      }
64      return addr;
65  }
```

Pagefault exception

```
$ vim kernel/trap.c
```

```
37 void
38 usertrap(void)
39 {
...
54     if (r_scause() == 8){
...
68         syscall();
69     } else if((which_dev = devintr()) != 0){
70         // ok
71     } else if((r_scause() == 15 || r_scause() == 13) &&
72             vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0) != 0) {
73         // page fault on lazily-allocated page
74     } else {
75         printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
76         printf("          sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
77         setkilled(p);
78     }
...
94 }
```

Exception Code	Description
8	Environment call from U-mode or VU-mode
12	Instruction page fault
13	Load page fault
15	Store/AMO page fault

vmfault

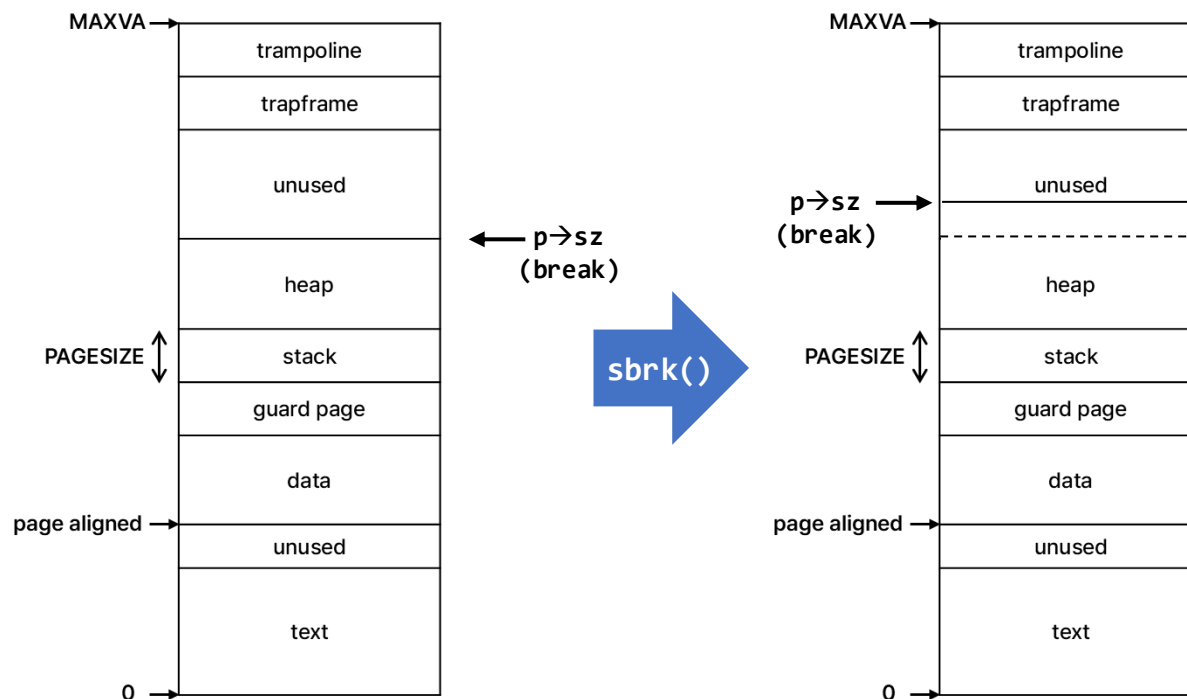
```
$ vim kernel/vm.c
```

```
452  uint64
453  vmfault(pagetable_t pagetable, uint64 va, int read)
454  {
455      uint64 mem;
456      struct proc *p = myproc();
457
458      if (va >= p->sz)
459          return 0;
460      va = PGROUNDDOWN(va);
461      if(ismapped(pagetable, va)) {
462          return 0;
463      }
464      mem = (uint64) kalloc();
465      if(mem == 0)
466          return 0;
467      memset((void *) mem, 0, PGSIZE);
468      if (mappages(p->pagetable, va, PGSIZE, mem, PTE_W|PTE_U|PTE_R) != 0) {
469          kfree((void *)mem);
470          return 0;
471      }
472      return mem;
473 }
```

1. Allocates a free physical page using `kalloc()`
2. Maps new physical page to virtual address

sbrk

- sbrk system call is used to dynamically adjust heap memory of the process.
- Provides lazy allocation which only updates the metadata pointer and defers physical allocation.



```
$ vim kernel/sysproc.c

39  uint64
40  sys_sbrk(void)
41  {
42      uint64 addr;
43      int t;
44      int n;
45
46      argint(0, &n);
47      argint(1, &t);
48      addr = myproc()->sz;
49
50      if(t == SBRK_EAGER || n < 0) {
51          if(growproc(n) < 0) {
52              return -1;
53          }
54      } else {
55          ...
62          myproc()->sz += n;
63      }
64      return addr;
65  }
```

growproc

```
$ vim kernel/proc.c
```

```
236 int
237 growproc(int n)
238 {
239     uint64 sz;
240     struct proc *p = myproc();
241
242     sz = p->sz;
243     if(n > 0){
244         if(sz + n > TRAPFRAME) {
245             return -1;
246         }
247         if((sz = uvmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
248             return -1;
249         }
250     } else if(n < 0){
251         sz = uvmdalloc(p->pagetable, sz, sz + n);
252     }
253     p->sz = sz;
254     return 0;
255 }
```

1. Gets new base address by fetching current heap size (p->sz)
2. Maps new physical page using `uvmalloc()` or frees them using `uvmdalloc()`
3. Updates p->sz to reflect the new total heap memory size

uvmalloc

```
$ vim kernel/vm.c
```

```
216  uint64
217  uvmalloc(pagetable_t pagetable, uint64 oldsz, uint64 newsz, int xperm)
218  {
219      char *mem;
220      uint64 a;
221
222      if(newsz < oldsz)
223          return oldsz;
224
225      oldsz = PGROUNDUP(oldsz);
226      for(a = oldsz; a < newsz; a += PGSIZE){
227          mem = kalloc();
228          if(mem == 0){
229              uvmdealloc(pagetable, a, oldsz);
230              return 0;
231          }
232          memset(mem, 0, PGSIZE);
233          if(mappages(pagetable, a, PGSIZE, (uint64)mem, PTE_R|PTE_U|xperm) != 0){
234              kfree(mem);
235              uvmdealloc(pagetable, a, oldsz);
236              return 0;
237          }
238      }
239      return newsz;
240  }
```

1. Expands user process's virtual memory space
 - Ensures addresses are page-aligned to prevent overlapping
2. Allocates physical memory and maps it to virtual addresses

uvmdealloc

```
$ vim kernel/vm.c
```

```
246  uint64
247  uvmdealloc(pagetable_t pagetable, uint64 oldsz, uint64 newsz)
248  {
249      if(newsz >= oldsz)
250          return oldsz;
251
252      if(PGROUNDUP(newsz) < PGROUNDUP(oldsz)){
253          int npages = (PGROUNDUP(oldsz) - PGROUNDUP(newsz)) / PGSIZE;
254          uvmunmap(pagetable, PGROUNDUP(newsz), npages, 1);
255      }
256
257      return newsz;
258  }
```

1. Calculates the number of pages to deallocate
 - Ensures addresses are page-aligned to prevent overlapping
2. Removes page table entries and frees physical memory

uvmunmap

```
$ vim kernel/vm.c
```

```
192 void
193 uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
194 {
195     uint64 a;
196     pte_t *pte;
197
198     if((va % PASIZE) != 0)
199         panic("uvmunmap: not aligned");
200
201     for(a = va; a < va + npages*PGSIZE; a+= PGSIZE){
202         if((pte = walk(pagetable, a, 0)) == 0) // leaf page...
203             continue;
204         if((*pte & PTE_V) == 0) // has physical...
205             continue;
206         if(do_free){
207             uint64 pa = PTE2PA(*pte);
208             kfree((void*)pa);
209         }
210         *pte = 0;
211     }
212 }
```

1. Locates the PTE for each virtual page using `walk()`
2. Frees the underlying page via `kfree()` if `do_free` is set
3. Clears the PTE to break the VA-to-PA mapping
4. Just continue if PTE does not exist (lazy-allocation)

uvmcopy

```
$ vim kernel/vm.c

296  uint64
297  uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
298  {
...
304      for(i = 0; i < sz; i += PGSIZE){
305          if((pte = walk(old, i, 0)) == 0)
306              panic("uvmcopy: pte should exist");
307          if((*pte & PTE_V) == 0)
308              panic("uvmcopy: page not present");
309          pa = PTE2PA(*pte);
310          flags = PTE_FLAGS(*pte);
311          if((mem = kalloc()) == 0)
312              goto err;
313          memmove(mem, (char*)pa, PGSIZE);
314          if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
315              kfree(mem);
316              goto err;
317          }
318      }
319      return 0;
320
321  err:
322      uvmunmap(new, 0, i / PGSIZE, 1);
323      return -1;
324  }
```

1. Finds the PA and flags from old page table
2. Allocates a new physical page for new page table
3. Copies Data using memmove()
4. Links the new physical page into new page table

Data Transfer Helper Functions

- **copyin(pagetable, dst, srcva, len)**
 - Copies data from user space (srcva) to kernel space.
- **copyinstr(pagetable, dst, srcva, max)**
 - Copies a null-terminated string from user space (srcva).
- **copyout(pagetable, dstva, src, len)**
 - Copies data from kernel space to user space (dstva).

Make Alarm System Call

Overview

- **Goal**

- Implement a feature that periodically alerts a process as it consumes CPU time.
- Add two new system calls: `sigalarm(interval, handler)` and `sigreturn()`

- **`sigalarm(int interval, void (*handler) ())`**

- If a user application calls `sigalarm(n, fn)`, then after every `n` ticks that the process consumes, the kernel should cause user application function `fn` to be called.
- If an application calls `sigalarm` with 0 interval, the kernel should stop generating periodic alarm calls.

- **`sigreturn()`**

- Assume that every handler function ends with a call to `sigreturn()`
- Restores the original context, resets the alarm state, and resumes the interrupted user application.

Setup for Hands-on labs

- **Branch Setup**

- Make a branch named 'sandbox/lab11' in sandbox repository

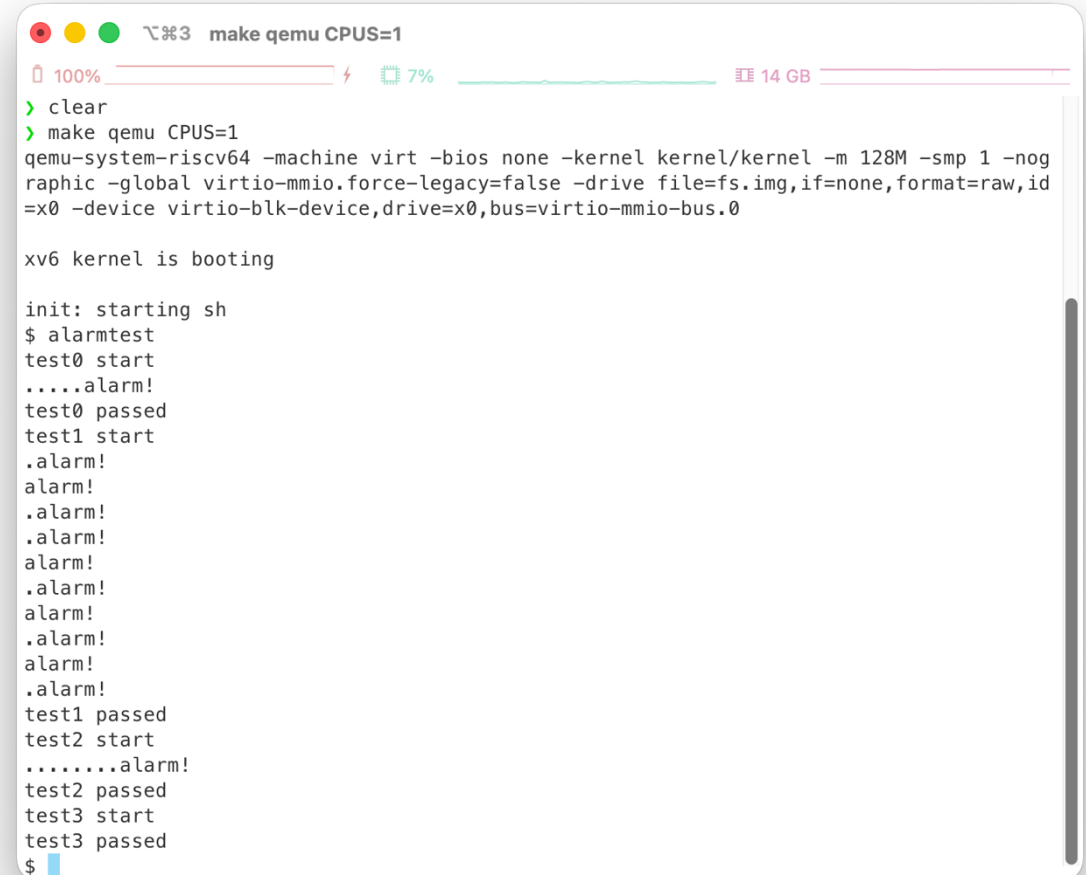
```
$ cd <path_to_your_own_sandbox_repo>      # move to your sandbox repository
$ git switch sandbox/base                  # switch to base branch
$ git branch sandbox/lab11                 # make a new branch
$ git switch sandbox/lab11                 # switch to lab11 branch
```

- **Skeleton Code and Solution**

- You can check the skeleton codes in [this commit](#).
- TA's code will be uploaded in [sandbox/lab11](#) branch in the xv6-sandbox repository.

Verifying Implementation

- Verify an implementation using `$ alarmtest`
 - test0: Basic alarm functionality
(Tick counting & Handler jump)
 - test1: Multi time handler execution
 - test2: Execution handler with Heavy workload
 - test3: Proper save/load user process' context



```
make qemu CPUS=1
100% 7% 14 GB
> clear
> make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ alarmtest
test0 start
....alarm!
test0 passed
test1 start
.alarm!
alarm!
.alarm!
alarm!
.alarm!
alarm!
.alarm!
alarm!
.alarm!
alarm!
test1 passed
test2 start
.....alarm!
test2 passed
test3 start
test3 passed
$
```

Thank you

